Bridging Scales through Generative AI for Inverse Problems in Biomedical Computational Microscopy

Mathematical Concepts, Inverse Problems, & VIRVS Virtual Staining Benchmark

Prof. Artur Yakimovich. Machine Learning for Infection and Disease Group

July 30, 2026

ayakimovich.github.io

Course Overview

1. Inverse Problems in Bioimaging
2. Distribution Learning & Probability
3. Generative Model Families
4. Virtual Staining Evaluation Metrics

SECTION 1

Inverse Problems in Bioimaging

Optical Physics, Microscope PSF Formation, Ill-Posedness & Autoencoders

Optical Physics: Microscope PSF Formation

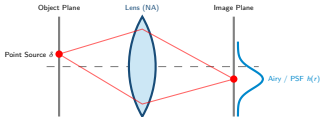
Physics of Impulse Response

- **Point Source**: A fluorophore emits spherical wavefronts $\delta(x, y)$.
- **Finite Aperture**: Objective lens with numerical aperture $\text{NA} = n \sin \alpha$ caps high spatial frequencies (low-pass filter).
- **Point Spread Function (PSF)**: Diffraction transforms a point source into an Airy pattern $h(r) \propto \left(\frac{J_1(kr)}{kr} \right)^2$.
- **Abbe Limit**: Minimum resolvable distance $d = \frac{\lambda}{2\text{NA}}$.

Biological Intuition (DAPI $\rightarrow$ GFP):

Single GFP fluorophores or fine DAPI chromatin granules are physically blurred by the microscope's PSF into smooth Airy disks. Generative models act as non-linear deconvolution solvers.

Microscope Optics & Code




```python
import torch

def generate_2d_gaussian_psf(size=15, sigma=2.0):
    # Coordinate grid
    coords = torch.arange(size) - size // 2
    y, x = torch.meshgrid(coords, coords, indexing='ij')

    # 2D Gaussian PSF approximation h(r)
    r_sq = x**2 + y**2
    psf = torch.exp(-r_sq / (2 * sigma**2))
    psf = psf / psf.sum() # Normalize integral to 1
    return psf.unsqueeze(0).unsqueeze(0)
```

Forward Problems in Bioimage Acquisition

Mathematical Formulation

Microscope image formation is a **forward problem**:

$$y = A(x) + n$$

- $x \in \mathbb{R}^N$: True biological signal (GFP reporter).
- A: Optical degradation (PSF blur, phase).
- $n \sim \mathcal{N}(0, \sigma^2 I)$: Detector noise.
- $y \in \mathbb{R}^M$: Observed micrograph (DAPI stain).

Biological Intuition (DAPI $\rightarrow$ GFP):

DAPI (y) reveals nuclear boundaries and chromatin texture, which are optical projections of cell state. GFP (x) marks positive expression. Microscope optics blur and add noise to these signals.

Python Code Equivalent

```python
import torch
import torch.nn.functional as F

def forward_observation(x, psf, sigma=0.05):
    # Blur image with Point Spread Function
    blurred = F.conv2d(x, psf, padding=1)

    # Add Gaussian detector noise n ~ N(0, sigma^2)
    noise = torch.randn_like(blurred) * sigma

    y = blurred + noise
    return y
```

Ill-Posedness of the Inverse Problem

Mathematical Formulation

Reconstructing x from y requires solving the **inverse problem**:

$$\hat{x} = A^{\dagger} y$$

Hadamard criteria for well-posedness:

1. **Existence**: Solution exists.
2. **Uniqueness**: Unique solution.
3. **Stability**: Continuous dependence on data y.

Bioimaging inverse problems are **severely ill-posed**: A is non-invertible/lossy!

Biological Intuition (DAPI $\rightarrow$ GFP):

Multiple cells may have identical nuclear shapes in DAPI, but some are GFP-positive and others negative. Inverting nuclear stain alone is ill-posed without spatial priors.

Python Code Equivalent

```python
# Naive inversion fails due to noise amplification

def naive_inversion(y, A_matrix):
    # SVD decomposition of measurement operator
    u, s, vh = torch.linalg.svd(A_matrix)

    # Small singular values amplify noise
    s_inv = torch.where(s > 1e-4, 1.0 / s, 0.0)
    A_pinv = vh.T @ torch.diag(s_inv) @ u.T

    x_hat = A_pinv @ y
    return x_hat
```

Classical Regularization (Tikhonov / Ridge)

MLID
machine learning for infection and disease
ayakimovich.github.io

Mathematical Formulation

To stabilize ill-posed inverse problems, we penalize implausible solutions:

$$\hat{x} = \arg\min_x \|y - Ax\|_2^2 + \lambda \|x\|_2^2$$

Closed-form solution:

$$\hat{x} = (A^T A + \lambda I)^{-1} A^T y$$

- First term: Data fidelity (observed y).
- Second term: Prior/regularizer (L_2 norm).
- $\lambda > 0$: Regularization strength.

Biological Intuition (DAPI → GFP):

Penalizes unrealistically extreme GFP intensity spikes outside cell boundaries, enforcing a smooth prior that fluorescence shouldn't explode randomly.

Python Code Equivalent

```python
def tikhonov_reconstruction(y, A, lmbda=1e-2):
    # A: [M, N] measurement matrix
    # y: [M, 1] observation vector
    N = A.shape[1]

    # System: (A^T A + lambda * I) x = A^T y
    AtA = A.T @ A
    regularizer = lmbda * torch.eye(N, device=A.device)

    lhs = AtA + regularizer
    rhs = A.T @ y

    x_hat = torch.linalg.solve(lhs, rhs)
    return x_hat
```

Autoencoders (AEs) & Representation Learning

MLID
machine learning for infection and disease
ayakimovich.github.io

Mathematical Formulation

Compression into a low-dimensional bottleneck $z \in \mathbb{R}^d$ ($d \ll N$):

$$z = e_\phi(x) \quad \text{(Encoder)}$$
$$\hat{x} = d_\theta(z) \quad \text{(Decoder)}$$

Reconstruction Loss: $\mathcal{L}_{\text{AE}} = \|x - d_\theta(e_\phi(x))\|_2^2$

Generative Sampling Limitation: Deterministic z-space has gaps and unconstrained regions. Random sampling $z \sim \mathcal{N}(0, I)$ produces unrealistic artifacts!

Biological Intuition (DAPI → GFP):

Standard AE compresses DAPI nuclear shape and GFP expression into bottleneck z. But because z-space has gaps, sampling random vectors yields non-biological cell artifacts.

Python Code Equivalent

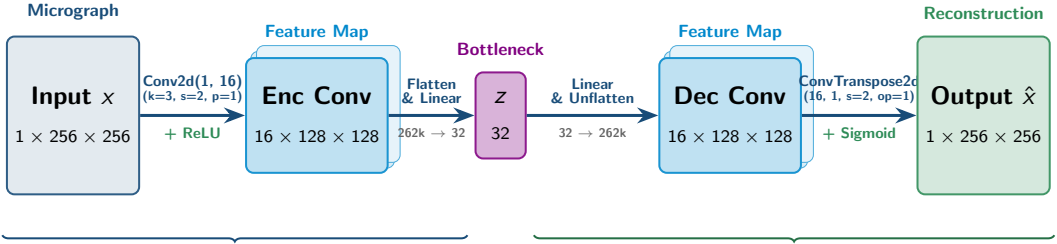
```python
import torch.nn as nn

class Autoencoder(nn.Module):
    def __init__(self, latent_dim=32):
        super().__init__()
        self.encoder = nn.Sequential(
            nn.Conv2d(1, 16, 3, stride=2, padding=1),
            nn.ReLU(),
            nn.Flatten(),
            nn.Linear(16 * 128 * 128, latent_dim)
        )
        self.decoder = nn.Sequential(
            nn.Linear(latent_dim, 16 * 128 * 128),
            nn.Unflatten(1, (16, 128, 128)),
            nn.ConvTranspose2d(16, 1, 3, stride=2, padding
                =1, output_padding=1),
            nn.Sigmoid()
        )

    def forward(self, x):
        z = self.encoder(x)
        return self.decoder(z)
```

Autoencoder (AE) Network Architecture & Tensor Dimensions

Layer-by-Layer Tensor Transformation (Matching PyTorch Implementation)



Architecture Summary & Key Takeaways

- **High Compression Ratio**: Reduces input dimension from $256 \times 256 = 65,536$ pixels down to $d = 32$ latent variables ($> 2,000\times$ compression ratio).
- **Symmetric Structural Flow**: Encoder contracts spatial dimensions ($256 \to 128$) while expanding channel depth ($1 \to 16$); Decoder mirrors this flow.
- **End-to-End Objective**: Optimized using Mean Squared Error $\|x - \hat{x}\|_2^2$ to force z to capture core morphological traits.

Mathematical Formulation

Map input x to latent distribution parameters $(\mu_\phi(x), \sigma_\phi(x))$:

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Objective (ELBO):

$$\mathcal{L}_{\text{VAE}} = \|x - \hat{x}\|_2^2 + \beta\, D_{\text{KL}}\left(q_\phi(z|x)\,||\,\mathcal{N}(0, I)\right)$$

Generative Sampling: KL penalty regularizes z-space to be smooth and gap-free. We can synthesize NEW valid cell images by sampling $z \sim \mathcal{N}(0, I)$!

Biological Intuition (DAPI → GFP):

VAE regularizes cell latent space so nuclear features and GFP expression vary smoothly. We can sample new synthetic DAPI/GFP cells or interpolate between GFP-negative and GFP-positive states!

Python Code Equivalent

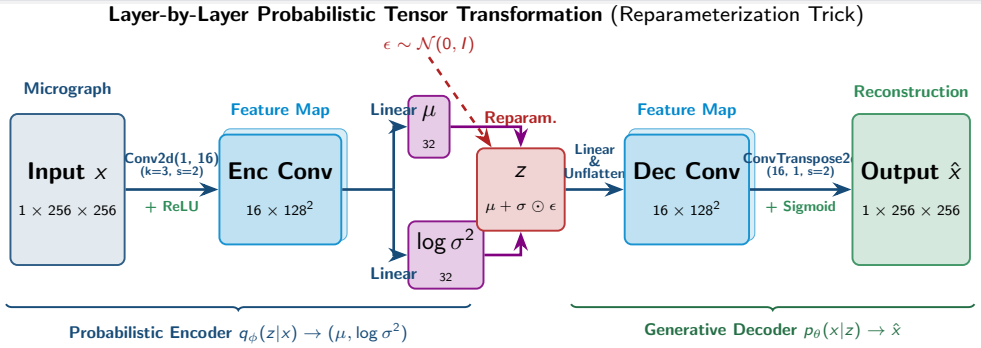
```
# Generative Sampling from Trained VAE Decoder

def generate_samples(decoder, num_samples=16, latent_dim
    =32, device='cpu'):
    decoder.eval()
    with torch.no_grad():
        # 1. Sample z ~ N(0, I) from standard Gaussian
            prior
        z_random = torch.randn(num_samples, latent_dim,
            device=device)

        # 2. Decode latent vectors into synthetic cell
            images
        x_synthetic = decoder(z_random)

    return x_synthetic # [num_samples, 1, H, W]
```

Variational Autoencoder (VAE) Architecture & Sampling



ELBO Objective & Generative Sampling

- **ELBO Loss**: $\mathcal{L}_{\text{VAE}} = \underbrace{\|x - \hat{x}\|_2^2}_{\text{Reconstruction Error}} + \beta \cdot \underbrace{D_{\text{KL}}\left(q_\phi(z|x) \,||\, \mathcal{N}(0, I)\right)}_{\text{KL Regularization Penalty}}$
- **Reparameterization Trick**: $z = \mu + \sigma \odot \epsilon$ allows backpropagation gradients to flow through stochastic sampling node!
- **Generative Synthesis**: To sample NEW synthetic cells, bypass Encoder and sample $z \sim \mathcal{N}(0, I) \rightarrow \text{Decoder}(z) \rightarrow \hat{x}_{\text{synthetic}}$.

Learning the Prior: Deep Inverse Problems

Mathematical Formulation

Instead of hand-crafted priors $\|x\|_2^2$ or $\mathrm{TV}(x)$, we learn a data-driven mapping f_θ:

$$\hat{x} = f_\theta(y) \approx \arg\min_x \Big(\|y - A(x)\|^2 + \lambda R_\theta(x) \Big)$$

Empirical Risk Minimization over training pairs (y_i, x_i):

$$\theta^* = \arg\min_\theta \frac{1}{N} \sum_{i=1}^{N} \mathcal{L}\big(f_\theta(y_i),\, x_i\big)$$

Biological Intuition (DAPI → GFP):

A deep neural network learns subtle nuclear features (e.g., nuclear swelling or DAPI texture changes) to predict whether a cell is GFP-positive or negative.

Python Code Equivalent

```python
import torch.nn as nn

class DeepInverseSolver(nn.Module):
    def __init__(self, backbone_unet):
        super().__init__()
        self.unet = backbone_unet # f_theta

    def forward(self, y):
        # Directly map degraded image y to reconstructed x
        return self.unet(y)

# Training objective over paired dataset
def compute_reconstruction_loss(x_pred, x_gt):
    return F.l1_loss(x_pred, x_gt)
```

Virtual Staining as a Non-Linear Inverse Problem

Mathematical & VIRVS Formulation

- **Input (y)**: DAPI/Hoechst nuclear micrograph.
- **Target (x)**: Fluorescent GFP reporter (> 0 for positive cell, 0 for negative).
- **Inverse Task** (Wyrzykowska et al., 2024):

$$x_{\text{GFP}} \approx f_\theta(y_{\text{DAPI}})$$

Biological Intuition (DAPI $\rightarrow$ GFP):
Predicts continuous GFP reporter intensity per cell directly from nuclear DAPI staining, enabling non-invasive quantification of positive vs negative cells!

Python Code Equivalent

```python
# Virus Infection Reporter Virtual Staining (VIRVS)
# Mapping Brightfield/DAPI y -> GFP Reporter x

def virvs_forward_step(model, dapi_img, gfp_target,
     optimizer):
    optimizer.zero_grad()

    # Predict continuous GFP fluorescence x_hat
    gfp_pred = model(dapi_img) # f_theta(y)

    # Mean Absolute Error Loss
    loss = F.l1_loss(gfp_pred, gfp_target)
    loss.backward()
    optimizer.step()

    return loss.item(), gfp_pred
```

SECTION 2

Distribution Learning & Probability

Density Estimation, Likelihood Maximization, KL Divergence & Wasserstein Distance

Empirical Data Distribution vs Parametric Density

Mathematical Formulation

Given observed image samples $\{x_1, x_2, \ldots, x_N\} \sim p_{\text{data}}(x)$:

$$p_{\text{data}}(x) = \frac{1}{N} \sum_{i=1}^{N} \delta(x - x_i)$$

Our goal is to learn a continuous parametric distribution $p_\theta(x)$ that approximates $p_{\text{data}}(x)$. Generative modeling = matching distributions:

$$p_\theta(x) \approx p_{\text{data}}(x)$$

Biological Intuition (DAPI → GFP):

p_{data} represents the real cell population containing a mixture of GFP-positive (bright) and GFP-negative (dark) cells. The model distribution p_θ must match this biological mixture.

Python Code Equivalent

```python
import torch
import torch.distributions as dist

# Empirical sample batch x_data
x_data = torch.randn(100, 1) * 2.0 + 5.0

# Parametric model distribution p_theta = N(mu, sigma)
mu = torch.tensor([0.0], requires_grad=True)
sigma = torch.tensor([1.0], requires_grad=True)

# Define parametric distribution family
p_theta = dist.Normal(loc=mu, scale=sigma)
```

Maximum Likelihood Estimation (MLE)

Mathematical Formulation

Finding optimal parameters θ^* by maximizing log-likelihood:

$$\theta^* = \arg\max_{\theta} \sum_{i=1}^{N} \log p_\theta(x_i)$$

Equivalent to minimizing Negative Log-Likelihood (NLL) / Forward KL:

$$\mathcal{L}_{\text{NLL}}(\theta) = -\mathbb{E}_{x \sim p_{\text{data}}} \left[\log p_\theta(x) \right]$$

Biological Intuition (DAPI → GFP):
Optimizes network parameters so that generating bright GFP signals for positive cells (and zero signal for negative cells) has maximum probability under true micrographs.

Python Code Equivalent

```python
def fit_mle(x_samples, p_theta_cls, lr=0.1, steps=100):
    mu = torch.tensor([0.0], requires_grad=True)
    sigma = torch.tensor([1.0], requires_grad=True)
    opt = torch.optim.Adam([mu, sigma], lr=lr)

    for _ in range(steps):
        opt.zero_grad()
        p_theta = p_theta_cls(loc=mu, scale=F.softplus(
            sigma))

        # Negative Log-Likelihood Loss
        nll_loss = -p_theta.log_prob(x_samples).mean()
        nll_loss.backward()
        opt.step()

    return mu, F.softplus(sigma)
```

Kullback-Leibler (KL) Divergence

Mathematical Formulation

Asymmetry measure of distance between two probability densities p and q:

$$D_{\mathrm{KL}}(p \,||\, q) = \int p(x) \log \frac{p(x)}{q(x)}\, dx$$

Analytical solution for two Gaussians $\mathcal{N}_0(\mu_0, \sigma_0^2)$ and $\mathcal{N}_1(\mu_1, \sigma_1^2)$:

$$D_{\mathrm{KL}}(p_0 \,||\, p_1) = \log \frac{\sigma_1}{\sigma_0} + \frac{\sigma_0^2 + (\mu_0 - \mu_1)^2}{2\sigma_1^2} - \frac{1}{2}$$

Biological Intuition (DAPI → GFP):

Quantifies information loss when approximating true GFP brightness distributions. Forward KL heavily penalizes predicting false GFP expression in cells that are actually negative.

Python Code Equivalent

```
# Analytical KL Divergence between 2 Gaussians in PyTorch

def kl_divergence_gaussians(mu0, log_var0, mu1=0.0,
     log_var1=0.0):
    # KL( N(mu0, var0) || N(mu1, var1) )
    var0 = torch.exp(log_var0)
    var1 = torch.exp(torch.tensor(log_var1))

    kl = 0.5 * (
        (log_var1 - log_var0) +
        (var0 + (mu0 - mu1)**2) / var1 -
        1.0
    )
    return kl.sum(dim=-1)
```

Wasserstein Distance (Optimal Transport)

Mathematical Formulation

Earth Mover's Distance (W_1 metric):

$$W_1(p, q) = \inf_{\gamma \in \Pi(p,q)} \mathbb{E}_{(x,y)\sim\gamma}\left[\|x - y\|\right]$$

Kantorovich-Rubinstein Duality:

$$W_1(p_{\text{data}}, p_\theta) = \sup_{\|f\|_L \leq 1} \mathbb{E}_{x\sim p_{\text{data}}}[f(x)] - \mathbb{E}_{x\sim p_\theta}[f(x)]$$

Key benefit: Smooth gradients even when support of p and q do not overlap!

Biological Intuition (DAPI $\rightarrow$ GFP):

Measures the minimum "work" needed to shift predicted cell GFP fluorescence profiles to match true biological GFP distributions across positive and negative populations.

Python Code Equivalent

```python
# 1D Empirical Wasserstein-1 Distance calculation

def compute_w1_distance(x_real, x_gen):
    # Sort samples along batch dimension
    x_real_sorted, _ = torch.sort(x_real, dim=0)
    x_gen_sorted, _ = torch.sort(x_gen, dim=0)

    # EMD in 1D is L1 distance between sorted quantiles
    w1_dist = torch.mean(torch.abs(x_real_sorted -
        x_gen_sorted))
    return w1_dist
```

SECTION 3

Generative Model Families

Variational Autoencoders (VAEs), Pix2Pix GANs & Score-based Diffusion Models

Variational Autoencoders (VAEs) & The ELBO

Mathematical Formulation

Evidence Lower Bound (ELBO):

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - D_{\mathrm{KL}}\left(q_\phi(z|x) \,||\, p(z)\right)$$

- **Likelihood**: Reconstruction fidelity (MSE/BCE).
- **KL Penalty**: Prior regularization ($p(z) = \mathcal{N}(0, I)$).

Loss to minimize:

$$\mathcal{L}_{\mathrm{VAE}}(\theta, \phi) = \mathcal{L}_{\mathrm{recon}} + \beta\, \mathcal{L}_{\mathrm{KL}}$$

Biological Intuition (DAPI $\rightarrow$ GFP):

Reconstruction aligns predicted GFP with DAPI nuclei; KL penalty forces latent infection states (GFP+ vs GFP-) into a smooth code space.

Python Code Equivalent

```python
def vae_loss_function(x_recon, x_target, mu, log_var, beta
    =1.0):
    # Reconstruction loss (MSE)
    recon_loss = F.mse_loss(x_recon, x_target, reduction='
        mean')

    # Analytical KL divergence against N(0, I)
    kl_loss = -0.5 * torch.mean(
        1 + log_var - mu.pow(2) - log_var.exp()
    )

    total_loss = recon_loss + beta * kl_loss
    return total_loss, recon_loss, kl_loss
```

The Reparameterization Trick

Mathematical Formulation

Sampling $z \sim q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \sigma^2_\phi(x)I)$ blocks backpropagation!

Solution: Isolate stochasticity into independent standard Gaussian noise ϵ:

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Gradients $\frac{\partial \mathcal{L}}{\partial \mu}$ and $\frac{\partial \mathcal{L}}{\partial \sigma}$ flow deterministically through encoder parameters ϕ.

Biological Intuition (DAPI $\rightarrow$ GFP):

Allows sampling continuous variations in GFP expression (e.g., dim vs bright positive cells) while enabling backpropagation gradients to pass cleanly through encoder weights.

Python Code Equivalent

```python
def reparameterize(mu, log_var):
    # Standard deviation sigma = exp(0.5 * log_var)
    std = torch.exp(0.5 * log_var)

    # Stochastic noise epsilon ~ N(0, I)
    eps = torch.randn_like(std)

    # Deterministic continuous transformation z
    z = mu + std * eps
    return z
```

Conditional GANs & Pix2Pix Virtual Staining

Mathematical Formulation

Conditional GAN minimax game for virtual staining ($y \to x$):

$$\min_G \max_D \mathbb{E}_{y,x}[\log D(y,x)] + \mathbb{E}_{y,z}[\log(1 - D(y, G(y,z)))]$$

Pix2Pix total generator loss (L_1 color/structure match):

$$\mathcal{L}_{\text{Pix2Pix}} = \mathcal{L}_{\text{cGAN}}(G,D) + \lambda \|x - G(y,z)\|_1$$

Biological Intuition (DAPI $\to$ GFP):

Generator synthesizes GFP patterns from DAPI; Discriminator acts like an expert microscopist checking if GFP co-localizes strictly with positive cells while staying 0 in negative cells.

Python Code Equivalent

```python
def pix2pix_generator_loss(D, G, y_brightfield, x_gt_fluo,
     lmbda=100.0):
    # Generate predicted virtual stain
    x_fake = G(y_brightfield)

    # Discriminator score on fake pair (y, G(y))
    pred_fake = D(y_brightfield, x_fake)

    # Adversarial Loss (wants D to predict 1)
    loss_gan = F.binary_cross_entropy_with_logits(
        pred_fake, torch.ones_like(pred_fake)
    )

    # L1 Reconstruction Loss
    loss_l1 = F.l1_loss(x_fake, x_gt_fluo)

    return loss_gan + lmbda * loss_l1
```

Denoising Diffusion Probabilistic Models (DDPM)

MLID machine learning for infection and disease
ayakimovich.github.io

Mathematical Formulation

Forward noise process adding Gaussian noise over timesteps t:

$$q(x_t|x_0) = \mathcal{N}\left(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)I\right)$$

Simplified training objective (ϵ-prediction MSE):

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t,x_0,\epsilon}\left[\|\epsilon - \epsilon_\theta(x_t, t)\|^2\right]$$

- $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon$
- $\epsilon_\theta(x_t, t)$: U-Net predicting added noise ϵ.

Biological Intuition (DAPI $\rightarrow$ GFP):
Gradually degrades GFP fluorescence into pure random noise, then learns to iteratively synthesize sharp GFP signals conditional on nuclear DAPI features.

Python Code Equivalent

```python
def ddpm_training_step(unet_noise_pred, x0, t, alpha_bar_t)
    :
    # 1. Sample noise epsilon ~ N(0, I)
    epsilon = torch.randn_like(x0)

    # 2. Compute noisy image x_t
    sqrt_alpha_bar = torch.sqrt(alpha_bar_t)
    sqrt_one_minus_alpha_bar = torch.sqrt(1.0 - alpha_bar_t
        )

    x_t = sqrt_alpha_bar * x0 + sqrt_one_minus_alpha_bar *
        epsilon

    # 3. Predict noise using U-Net
    epsilon_pred = unet_noise_pred(x_t, t)

    # 4. MSE Loss on noise
    loss = F.mse_loss(epsilon_pred, epsilon)
    return loss
```

SECTION 4

Virtual Staining Evaluation Metrics

VIRVS Benchmark: PSNR, SSIM, PCC, MAE & Cell-level Reporter Quantification

Pixel-Wise Metrics: PSNR & MAE

Mathematical Formulation

Mean Absolute Error (MAE):

$$\text{MAE}(x, \hat{x}) = \frac{1}{N} \sum_{i=1}^{N} |x_i - \hat{x}_i|$$

Peak Signal-to-Noise Ratio (PSNR in dB):

$$\text{PSNR}(x, \hat{x}) = 10 \cdot \log_{10} \left(\frac{\text{MAX}_x^2}{\text{MSE}(x, \hat{x})} \right)$$

where $\text{MSE}(x, \hat{x}) = \frac{1}{N} \sum_{i=1}^{N} (x_i - \hat{x}_i)^2$.

Biological Intuition (DAPI → GFP):

MAE measures average GFP intensity error per pixel. Lower MAE means correctly predicting 0 GFP in negative cells and accurate brightness in GFP-positive cells.

Python Code Equivalent

```python
def compute_psnr_mae(x_gt, x_pred, max_val=1.0):
    # MAE Calculation
    mae = torch.mean(torch.abs(x_gt - x_pred))

    # MSE & PSNR Calculation
    mse = torch.mean((x_gt - x_pred) ** 2)
    if mse == 0:
        psnr = torch.tensor(float('inf'))
    else:
        psnr = 10.0 * torch.log10((max_val ** 2) / mse)

    return psnr.item(), mae.item()
```

Structural Similarity Index (SSIM)

Mathematical Formulation

SSIM models human visual perception (luminance, contrast, structure):

$$\text{SSIM}(x, \hat{x}) = \frac{(2\mu_x\mu_{\hat{x}} + C_1)(2\sigma_{x\hat{x}} + C_2)}{(\mu_x^2 + \mu_{\hat{x}}^2 + C_1)(\sigma_x^2 + \sigma_{\hat{x}}^2 + C_2)}$$

- $\mu_x, \mu_{\hat{x}}$: Local mean intensities.
- $\sigma_x^2, \sigma_{\hat{x}}^2$: Local variances.
- $\sigma_{x\hat{x}}$: Local covariance.
- C_1, C_2: Stabilization constants.

Biological Intuition (DAPI → GFP):

Evaluates whether predicted GFP spatial boundaries match true cell shapes around DAPI-stained nuclei, rather than just checking overall background brightness.

Python Code Equivalent

```python
from skimage.metrics import structural_similarity as ssim

def compute_ssim_numpy(x_gt_np, x_pred_np):
    # x_gt_np, x_pred_np: [H, W] float32 arrays in [0, 1]
    ssim_score = ssim(
        x_gt_np,
        x_pred_np,
        data_range=1.0,
        win_size=7
    )
    return ssim_score
```

Pearson Correlation Coefficient (PCC)

Mathematical Formulation

Linear correlation between predicted fluorescence $\hat{x}$ and ground truth x:

$$\mathrm{PCC}(x, \hat{x}) = \frac{\sum_{i=1}^{N}(x_i - \bar{x})(\hat{x}_i - \bar{\hat{x}})}{\sqrt{\sum_{i=1}^{N}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{N}(\hat{x}_i - \bar{\hat{x}})^2}}$$

- Range: $[-1, 1]$.
- $\mathrm{PCC} = 1$: Perfect linear relationship.
- Insensitive to global linear intensity scaling/shifts!

Biological Intuition (DAPI $\rightarrow$ GFP):
Checks if relative GFP intensity rankings across cells match truth (strongly positive vs weakly positive cells) regardless of overall brightness scaling.

Python Code Equivalent

```python
def compute_pcc(x_gt, x_pred):
    # Flatten spatial dimensions
    x1 = x_gt.flatten()
    x2 = x_pred.flatten()

    # Zero-mean vectors
    x1_m = x1 - torch.mean(x1)
    x2_m = x2 - torch.mean(x2)

    # Covariance divided by product of standard deviations
    num = torch.sum(x1_m * x2_m)
    den = torch.sqrt(torch.sum(x1_m ** 2) * torch.sum(x2_m
        ** 2) + 1e-8)

    pcc = num / den
    return pcc.item()
```

Biological Validity: Single-Cell Infection Quantification

VIRVS Biological Metric

Virtual staining models must preserve biologically meaningful signal quantification!

For single-cell binary mask $M_k \in \{0,1\}^{H \times W}$:

$$I_{\text{cell},k} = \frac{1}{\sum_{ij} M_{k,ij}} \sum_{i,j} M_{k,ij} \cdot x_{ij}$$

Benchmark compares true single-cell viral intensity $I_{\text{true},k}$ vs predicted $I_{\text{pred},k}$ across infection states.

Biological Intuition (DAPI → GFP):

Directly quantifies mean GFP reporter signal within each DAPI-segmented cell mask to classify individual cells as infected (GFP+) or uninfected (GFP-).

Python Code Equivalent

```python
def compute_cell_level_viral_reporter(x_fluo, cell_masks):
    # cell_masks: [Num_Cells, H, W] boolean array
    cell_intensities = []

    for k in range(cell_masks.shape[0]):
        mask = cell_masks[k]
        # Mean fluorescence intensity inside cell mask k
        cell_intensity = (x_fluo * mask).sum() / (mask.sum
            () + 1e-8)
        cell_intensities.append(cell_intensity)

    return torch.tensor(cell_intensities)
```

Summary & Workshop Transition

Key Takeaways

1. **Inverse Problems**: Virtual Staining maps label-free micrographs y to continuous virus infection reporter fluorescence x.
2. **Distribution Learning**: Generative models minimize divergence (NLL, KL, Wasserstein) between empirical data p_{data} and model p_θ.
3. **Model Families**: VAEs (ELBO), Pix2Pix GANs (cGAN + L_1), and Diffusion Models (ϵ-prediction).
4. **VIRVS Benchmarking**: Multi-faceted evaluation combining PSNR, SSIM, PCC, MAE, and cell-level viral reporter quantification.

Next Up: Practical VIRVS Benchmark Notebooks!